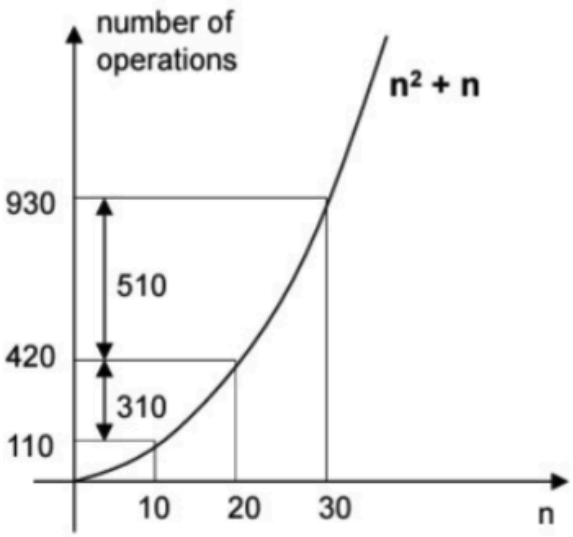
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/DSA	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct. 7, 2026	**DATE SUBMITTED:**	Oct. 7, 2026
NAME:	Formalejo, Francis Elijah Ramirez, Angel Mae	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

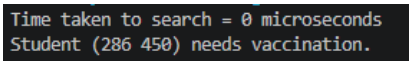
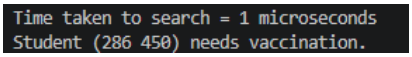
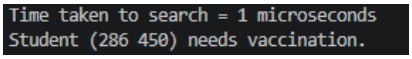


n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

For our analysis, we calculated the worst case scenario by multiplying the length of array x by the length of array y. To show this, let m = x length and n = y length. Our outer loop runs n times, while our search function scans all m elements in array x when no match exists. Therefore, total worst case comparisons equal **m x n**.

When array x and array y have equal length, we set m equal to n. Consequently, our operation count becomes T of n = n^2 + n. As a result, our algorithm belongs to the quadratic time complexity class, and our graph forms a rising curve. This means doubling our input size quadruples our total operations. Our input size 10 = 110 operations, input size 20 = 420 operations, and input size 30 = 930 operations. Overall, our nested loops scale poorly as our dataset grows.

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	0, 0, 0 microseconds		Searching 1,000 elements consistently takes 0 microseconds across all runs because binary search requires only about 10 comparison steps
10,000	1, 1, 1 microseconds		Increasing our dataset by a factor of 10 increases search execution time to 1 microsecond
100,000	2, 3, 5 microseconds		We increased our dataset size 100 times that results to have small time variations between 2 to 5 microseconds

Analysis: Growing our list 100 fold from 1,000 to 100,000 raises execution time from under 1 to a few microseconds, while our iteration count rises from about 10 to about 17 steps. Our step count matches log2(N) closely, proving the O(log N) complexity of binary search. Multiplying N by 10 adds only about 3.3 comparison steps. A linear search requires up to 100,000 comparisons. Execution time grows slightly faster than the step count at large N because the larger list no longer fits inside CPU cache memory. Exact microseconds vary across runs and hardware, but our growth pattern remains constant

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

CODE

problem1.cpp > main()

```cpp

#include <iostream>
using namespace std;

bool Unique(int A[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

int main() {
    int A[] = {1, 2, 3, 4, 5};
    int n = sizeof(A) / sizeof(A[0]);

    if (Unique(A, n))
        cout << "true" << endl;
    else
        cout << "false" << endl;

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\aloha> C:\Users\TIPQC\Documents\aloha\problem1.exe
true
PS C:\Users\TIPQC\Documents\aloha>
```

ANALYSIS:
RAMIREZ:
The output is true because all the elements in the array are unique. The program compares each element with the elements after it and does not find any duplicate values. If a duplicate is found, the function returns false.
FORMALEJO:
The code compares 1 to every other number, then 2 to the rest, and keeps checking each pair. If It finds no matching numbers while checking the list because it finds no duplicates, therefore the code's output is true

THEORETICAL ANALYSIS:
RAMIREZ:
The algorithm uses two nested loops to compare the elements. In the worst case, when all elements are unique, the number of comparisons is C(n)= n(n−1)/2. Therefore, the worst-case time complexity is O(n^2). The best-case time complexity is O(1), when the first comparison finds a duplicate. The auxiliary space complexity is O(1).
FORMALEJO:
The code uses two loops to check pairs of numbers which are: The outer loop that selects a number, and the inner loop that compares it to every remaining number. When all numbers are unique, the code performs N x N -1 / 2. This creates quadratic time complexity, written as O(N^2). If the first two numbers match, the code stops early in O(1) time. My code keeps space complexity at O(1) because it uses fixed memory variables without allocating new memory

EXPERIMENTAL ANALYSIS:
RAMIREZ:
The program was designed to test an array containing five unique elements. Its expected output is true. Since this example uses only five elements, the program performs a small number of comparisons. Actual execution

time should be measured on the computer used for the experiment.
FORMALEJO:
The algorithm checks every pair to confirm no duplicate values exist. It was true because the test array contains very few items. You can also measure actual runtimes in microseconds on your local machine using the chrono

PROBLEM 2
CODE

problem2.cpp > brpower(long long, int)

```cpp

#include <iostream>
using namespace std;

// Regular recursive power
long long rpower(long long x, int n) {
    if (n == 0)
        return 1;

    return x * rpower(x, n - 1);
}

// Binary recursive power
long long brpower(long long x, int n) {
    if (n == 0)
        return 1;

    long long y;

    if (n % 2 == 1) {
        y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    y = brpower(x, n / 2);
    return y * y;
}

int main() {
    cout << "rpower(2, 10) = "
         << rpower(2, 10) << endl;

    cout << "brpower(2, 10) = "
         << brpower(2, 10) << endl;

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\aloha> & C:\Users\TIPQC\Documents\aloha\problem2.exe
rpower(2, 10) = 1024
brpower(2, 10) = 1024
PS C:\Users\TIPQC\Documents\aloha> & C:\Users\TIPQC\Documents\aloha\problem2.exe
```

ANALYSIS:
RAMIREZ:
The output shows that both rpower(2,10) and brpower(2,10) return 1024, which is the correct value of 2^10 . Although both functions produce the same answer, they use different methods to calculate the power.
FORMALEJO:
For an exponent of 1000, rpower needs 1000 steps while brpower needs only 10 steps. The brpower function saves time and memory and the brpower function is much faster than rpower

THEORETICAL ANALYSIS:
RAMIREZ:
The rpower algorithm reduces the exponent by one in every recursive call. Its recurrence relation is: $T(n)=T(n-1)+O(1)$ Therefore, its time complexity is O(n), and its auxiliary space complexity is O(n). The brpower algorithm reduces the exponent by approximately half in every recursive call. Its recurrence relation is $T(n)=T(n/2)+O(1)$ Therefore, its time complexity is O(logn), and its auxiliary space complexity is O(logn).
FORMALEJO:
The rpower function reduces the exponent n by 1 in every step. Then it creates a chain of n loop calls, giving it a linear time complexity of O(n). The brpower function uses repeated multiplication by dividing the exponent n by 2 at each step. It makes log2(n) nested calls, giving it a logarithmic time complexity of O(log n). The memory stack grows to O(n) for rpower and stays at O(log n) for brpower

EXPERIMENTAL ANALYSIS:
RAMIREZ:
The program calculates 2102^{10}210 using both algorithms. The output shows that both functions return 1024, confirming that they produce the correct result for this input. This test verifies the result but does not, by itself, prove which algorithm is faster. To compare their actual execution times, the program must measure both

functions using a timer and repeat the tests with larger exponents.
FORMALEJO:
Both functions print 1024 because the rpower function makes 10 recursive calls to complete the math. The brpower function finishes in 4 recursive steps. The test uses small input values, so both functions complete almost instantly on local hardware

CONCLUSION:
RAMIREZ:
This hoa helped me understand how theoretical and experimental analysis can be used to evaluate algorithms. In Problem 1, I saw that the Unique() function has a worst-case time complexity of O(n^2) because it compares pairs of elements using nested loops. In Problem 2, I learned that rpower has O(n) time complexity, while brpower has O(logn) time complexity because it reduces the exponent by half in each recursive call.The results also showed that two algorithms can produce the same correct answer while having different levels of efficiency, this hoa definity helped me understand why choosing an efficient algorithm is just as important, especially when working with some larger inputs
FORMALEJO:
This HOA taught me on how algorithm design works, like for problem 1, the algorithm relies on nested loops to find matching pairs, resulting in quadratic O(N^2). And in problem 2, the rpower function processes exponents linearly in O(N) steps, while my brpower function cuts problem size in half to finish in logarithmic O(log N) steps. Overall, this was a 7.5/10 for me since the analyzing takes some time and I have to review back and forth to understand it

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.